



2026년 5월 기준 llm wiki claude code나 codex에 붙이는 스킬 중 어떤 레포가 젤 유망하며 루틴이 어떻게 돼?

이제 충분한 정보가 모였습니다. 정리해드리겠습니다.

LLM Wiki 스킬: 개요

LLM Wiki는 Andrej Karpathy가 2026년 4월 초 공개한 [GitHub Gist](#)에서 시작된 패턴으로, Claude Code / Codex에 붙이는 스킬 중 현재 가장 화제의 카테고리입니다. RAG처럼 매번 원본 문서에서 답을 재조합하는 게 아니라, LLM이 직접 마크다운 **Wiki**를 쓰고 누적 유지하는 구조가 핵심입니다.

가장 유명한 레포 비교

레포	Stars	특징	지원 에이전트
nvk/llm-wiki	★ 294	가장 기능이 풍부, 병렬 멀티에이전트, 클로드 퍼스트 설계	Claude Code, Codex, OpenCode, Pi
llm-wiki-agent (SkillsLLM 집계)	★ 2,653	개인 지식베이스 자동 유지, 멀티 에이전트 지원	Claude Code, Codex, Gemini
Pratiyush/llm-wiki	(활성 개발 중)	Python CLI(llmwiki) 포함, session history 기반 자동 수집	Claude Code, Codex CLI, Copilot, Cursor, Gemini
dair-ai/wiki-builder	(plugin 마켓 배포)	원커맨드 Wiki 스캐폴딩, research/paper/product 플레이버 지원	Claude Code
iamsashank09/llm-wiki-kit	(활성)	MCP 서버 기반, PDF/URL/YouTube 자동 수집, 세션 간 기억 유지	Codex + MCP 호환 전체

nvk/llm-wiki – 가장 완성도 높은 루틴

현재 Claude Code + Codex 양쪽을 공식 지원하며 가장 체계적인 레포입니다. v0.4.4 기준(2026-04-26) 기능이 정리되어 있습니다.

설치

```
# Claude Code
claude plugin install wiki@llm-wiki

# OpenAI Codex
```

```
codex plugin marketplace add nvk/llm-wiki
# → Codex에서 /plugins 열고 "LLM Wiki" 활성화 후 @wiki 사용
```

기본 루틴 (워크플로우)

핵심은 **Research** → **Ingest** → **Compile** → **Query** → **Lint** → **Output** 6단계 사이클입니다.

1. Init — 토픽별 독립 Wiki 생성

```
/wiki init nutrition          # 일반 토픽
/wiki init my-project --local # 프로젝트 로컬 .wiki/
```

2. Research — 병렬 에이전트가 자동 수집·컴파일

```
/wiki:research "AI agents" --new-topic      # 토픽 생성 + 즉시 리서치
/wiki:research "RAG vs wiki" --deep         # 8 에이전트 딥모드
/wiki:research "LLM memory" --min-time 1h   # 1시간 동안 라운드 반복
```

3. Ingest — URL, 파일, 텍스트 수동 추가

```
/wiki add https://arxiv.org/abs/...
/wiki:ingest --inbox # inbox/ 폴더 일괄 처리
```

4. Compile — raw 소스 → wiki 아티클 합성

```
/wiki:compile
/wiki:compile --full # 처음부터 전체 재컴파일
```

5. Query — wiki에 질문

```
/wiki:query "How does fiber affect mood?"
/wiki:query "compare keto vs mediterranean" --deep
/wiki:query --resume # 세션 재개 시 컨텍스트 복원
```

6. Lint + Output — 품질 관리 및 산출물 생성

```
/wiki:lint --fix
/wiki:librarian # 오래된/품질 낮은 아티클 탐지
/wiki:output report --topic my-project
/wiki:ll # 현 세션의 lessons learned 추출·저장
```

Research 모드별 에이전트 수

모드	플래그	에이전트 수
Standard	(기본)	5개 (학술·기술·적용·뉴스·반론)
Deep	--deep	8개 (+ 역사·인접분야·통계)
Retardmax	--retardmax	10개 (분석 생략, 최대 속도)

핵심 설계 원칙 (Karpathy 원본 개념)

- Wiki = 영속 아티팩트 — RAG은 매번 재발견, Wiki는 지식이 누적됨
- 3레이어 구조 — raw/(불변 원본) → wiki/(LLM 작성 마크다운) → schema(CLAUDE.md / AGENTS.md)
- LLM이 유지보수자 — 크로스 레퍼런스·모순 탐지·페이지 갱신을 LLM이 자동 처리, 인간은 소싱과 방향 지시에 집중
- Obsidian 연동 — [[wikilinks]] + 표준 마크다운 링크 동시 사용으로 어떤 뷰어에서도 동작

주의사항 (실전 팁)

CLAUDE.md / AGENTS.md 자체는 짧고 직접 작성한 것이 효과적입니다. LLM이 자동 생성한 컨텍스트 파일은 오히려 성능을 소폭 저하시킨다는 실험 결과도 있으며, 200줄 이하로 유지하는 게 권장됩니다. Codex용으로는 @wiki 호출이 표준이며 첫 설치 후 /plugins 화면에서 수동 활성화가 필요합니다.

✱

<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

여기가 원본 gist llm wiki고

아래 댓글 코멘트도 엄청 많은데 한 번 검토해줘봐

원본 Gist + 댓글 전체를 다 읽었습니다. 핵심 내용과 주목할 프로젝트들을 정리해드립니다.

원본 Gist 핵심 요약

Gist는 별 5,000+, Fork 5,000+ 를 달성한 상태입니다. 의도적으로 추상적인 아이디어 문서로 작성했으며 특정 구현을 강제하지 않습니다 — "이 패턴을 에이전트에게 붙여넣으면 에이전트가 너와 협력해서 구체화한다"는 컨셉입니다.

3레이어 구조 :

- raw/ — 불변 원본 소스 (LLM은 읽기만)
- wiki/ — LLM이 직접 쓰는 마크다운 파일 전체
- schema (CLAUDE.md / AGENTS.md) — 워크플로우 규칙, LLM이 generic chatbot이 아닌 disciplined wiki maintainer가 되게 하는 설정

4가지 핵심 오퍼레이션 :

1. Ingest — 소스 드롭 → 읽기 → 요약 → 10~15개 위키 페이지 업데이트 → 로그 append
2. Query — 인덱스 읽기 → 관련 페이지 탐색 → 답변 + 좋은 답변은 wiki에 파일로 저장
3. Lint — 주기적 건강검진 (모순 탐지, 고아 페이지, 오래된 클레임, 누락 크로스레퍼런스)
4. index.md + log.md — 인덱스(컨텐츠 카탈로그)와 로그(시간순 기록) 두 파일로 탐색

Obsidian을 "IDE", LLM을 "programmer", wiki를 "codebase"로 비유한 것이 핵심 프레임입니다 .

댓글에서 주목할 프로젝트들

댓글이 완전히 생태계 쇼케이스가 됐습니다. 실질적으로 쓸만한 것만 골라 정리합니다.

즉시 써볼 수 있는 구현체

프로젝트	Stars	특징
skyllwt/OmegaWiki	740+ ★ (v1.5.0)	논문 특화, 26개 Claude Code 스킬, discover→ingest→ideate→experiment→draft→poster→rebuttal 풀 사이클
kytmanov/synto	(obsidian- llm-wiki 후속)	Local-first, Ollama/LM Studio 지원, MCP 서버 8개 톨, 초안→검증→발행 3단계 draft 상태 관리
samethbrr/llm-wiki- manager	-	Claude Code 스킬로 바로 설치 (git clone ... ~/ .claude/skills/), 8가지 오퍼레이션 모드, 4개 Python 스크립트
gowtham0992/link	-	MCP 기반, Brew 설치, Codex/Claude/Cursor/Kiro 전부 지원, SQLite FTS 검색
Electro- resonance/LLM-WIKI- MCP	-	MCP 서버 + CLI, Ollama 로컬 모델 지원, 파일 해시로 변경분만 재수집
ojuschugh1/sqz	-	LLM Wiki 전용은 아니지만 토큰 압축 도구, 같은 파일 5회 읽기 시 92% 절감, Claude Code PreToolUse 혹은 투명하게 동작

특수 목적 구현체

- WayneChou-bot/LLM-Wiki-Agent-Workflow-Demo — 4개 에이전트 역할(프로그래밍/UI/PM/개인지식)이 같은 소스를 각자 시각에서 컴파일, Streamlit 데모
- axoviq-ai/synthadoc — Adversarial Review(두 번째 LLM이 모든 페이지 독립 검증) + Claim-Level Provenance(단락마다 원본 라인 추적)
- eslamgenio/long-term-agent-memory — 세션 간 결정/절차/연결지식 영속 저장
- ek0212/second-brain-template — Obsidian 세컨드 브레인 특화 템플릿

댓글에서 나온 실전 문제점들

댓글 중 실제 사용자가 한 달 이상 써보고 발견한 문제들이 더 가치 있습니다 :

1. Identity 문제 (timfong888) — 같은 개념이 약간 다른 이름으로 중복 페이지 생성됨. 예: "Cognitive Dissonance Marketing" vs "Cognitive Dissonance and Urgency"
2. Level 문제 — 거시 테마("Personal AGI")와 전술적 메모("Urgency Trigger")가 같은 레벨로 플랫폼
하게 쌓임. 중요도 구분이 사라짐
3. Relationship 문제 — 모든 링크가 "related"로만 표현됨. Similar / Contains / Contradicts 구분 없
이 그래프가 탐색용으로만 기능하고 사고 도구로는 부족함

4. **Staleness** 문제 (dtmoura) – CLAUDE.md에 "업데이트 잊지 마" 써도 에이전트가 자동으로 wiki를 갱신하지 않음. 정확한 프롬프트 없이는 wiki가 낡아짐
5. 스케일링 문제 (ahumanft) – wiki가 커질수록 인제스트/스키마/역할/검색 각 컴포넌트를 좁게 분리하지 않으면 LLM이 오버로드됨 → **Segmentation** 패턴(V3)으로 대응

sqz – 함께 쓰면 좋은 토큰 압축 툴

LLM Wiki를 쓰면 같은 파일을 세션 중 여러 번 읽게 됩니다. sqz는 PreToolUse 훅으로 Claude Code에 투명하게 붙어서 동일 파일 반복 읽기 시 92% 토큰 절감을 실현합니다. Rust 단일 바이너리라 설치도 간단합니다:

```
curl -fsSL https://raw.githubusercontent.com/ojuschugh1/sqz/main/install.sh | sh
sqz init --global
```

영재님 프로젝트들처럼 여러 파일을 반복적으로 참조하는 agentic 세션에서 특히 효과적입니다.



1. <https://mcpmarket.com/tools/skills/llm-friendly-documentation-generator>
2. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f?permalink_comment_id=6085265
3. <https://calv.info/agents-feb-2026>
4. https://www.linkedin.com/posts/stevenc81_i-built-a-claude-code-skill-called-truth-seeking-activity-7453291402899193856-qHzY
5. <https://skillsllm.com/skill/llm-wiki>
6. <https://blakecrosley.com/ko/blog/agents-md-patterns>
7. <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>
8. https://tool.lu/en_US/library/40J
9. https://www.reddit.com/r/ClaudeCode/comments/1sm374u/turned_andrej_karpathys_llm_wiki_gist_into_a/
10. <https://mcpmarket.com/ko/daily/skills/top-skill-list-march-23-2026>
11. <https://x.com/nvk/status/2040785527419400324>
12. <https://agents.md>
13. https://www.reddit.com/r/codex/comments/1sgen4x/making_codex_remember_across_sessions_built_an/
14. https://www.reddit.com/r/claude/comments/1s51b5u/the_claude_code_skills_actually_worth_installing/
15. <https://www.youtube.com/watch?v=9iWTRMjbBvo>
16. <https://github.com/travisvn/awesome-claude-skills>
17. <https://dev.to/rosidotidev/karpathys-llm-wiki-no-code-with-claude-or-github-copilot-5fb0>
18. <https://github.com/topics/claude-code-skills>
19. <https://hackernoon.com/how-i-built-a-self-maintaining-knowledge-base-for-6-projects-using-claude-code-and-karpathys-llm-wiki>
20. <https://composio.dev/content/top-claude-skills>

21. <https://skillsllm.com/compare/llm-wiki-agent-vs-ui-ux-pro-max-skill>
22. <https://velog.io/@aiengineer/2026년-2월-GitHub-트렌드-TOP10-AI-에이전트-생태계가-개발-표준이-되다>
23. <https://academy.dair.ai/blog/wiki-builder-claude-code-plugin>
24. <https://x.com/NousResearch/status/2041378745332961462>
25. <https://github.com/topics/llm-wiki-personal-knowledge-base>
26. <https://www.youtube.com/watch?v=v8rCHym0lXE>
27. <https://github.com/lewislulu/llm-wiki-skill>
28. <https://skillsllm.com/compare/mduongvandinh-llm-wiki-vs-ui-ux-pro-max-skill>
29. <https://github.com/nvk/llm-wiki/issues>
30. <https://www.mindstudio.ai/blog/what-is-llm-wiki-karpathy-knowledge-base-architecture/>
31. <https://gist.github.com/joonan30/f6013d9da55edb506360b2fcb588997c>
32. <https://github.com/Pratiyush/llm-wiki/blob/master/docs/tutorials/01-installation.md>
33. https://www.reddit.com/r/ClaudeCode/comments/1ssimaz/best_github_repos_for_claude_code/
34. <https://github.com/Pratiyush/llm-wiki>
35. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f?permalink_comment_id=6086902
36. <https://github.com/BehiSecc/awesome-claude-skills>
37. <https://github.com/Pratiyush/llm-wiki/discussions>
38. <https://github.com/ComposioHQ/awesome-claude-skills>
39. <https://pratiyush.github.io/llm-wiki/>
40. <https://buldr.com/top-60-claude-skills-workflows-github-repos-for-ai/>
41. <https://github.com/Pratiyush/llm-wiki/blob/master/.llmwiki-synth-state.json>
42. <https://www.kdnuggets.com/10-github-repositories-to-master-claude-code>
43. <https://github.com/Pratiyush/llm-wiki/issues>
44. <https://www.ayautomate.com/blog/best-claude-code-github-repos>
45. <https://github.com/anthropics/claude-code/issues/6235>
46. <https://github.com/Pratiyush/llm-wiki/discussions/categories/general>
47. https://www.reddit.com/r/ClaudeCode/comments/1scidpz/built_a_claude_code_plugin_that_turns_your/
48. <https://roboco.io/posts/karpathy-llm-wiki-72-run-benchmark/>
49. <https://zazencodes.substack.com/p/stop-using-agentsmd-and-claudemd>
50. <https://github.com/nvk/llm-wiki>
51. paste.txt